

A-N CONTENT GENERATOR

Multi-Agent Content Generation System with AI

Technical Documentation - June 2026

1. Executive Summary

Content Generator POC is a digital content generation platform powered by Artificial Intelligence. Its purpose is to automate the creation of optimized posts for different social networks and content formats (blogs, Twitter/X, Instagram, LinkedIn, scientific articles, and financial analysis), and publish them directly to those platforms without manual intervention.

The system is built on a specialized microservices architecture, where each component has a unique and well-defined responsibility. This architecture is known as a **multi-agent architecture** because each service acts as an intelligent agent that performs a specific task, communicates with others through clear interfaces, and can operate autonomously.

Why a multi-agent architecture?

- **Scalability:** Each agent can scale independently according to demand.
- **Maintainability:** If an agent fails or needs to be updated, it does not affect the rest of the system.
- **Specialization:** Each agent is an expert in its area (generation, publication, data retrieval).
- **Resilience:** The system implements automatic fallback chains to guarantee availability.

2. General System Architecture

The system is organized into clearly differentiated layers. The entry layer receives user requests through the REST API. The orchestration layer coordinates the different agents according to the type of content requested. The generation layer produces the content using language models. The data layer manages persistent storage. Finally, the publication layer distributes the content to the target platforms.

2.1 Directory Structure

Folder / File	Purpose
<code>services/agent_orchestrator/</code>	Coordinates the flow between all agents according to the type of request.
<code>services/content_generator/</code>	Generation core: prompts, data schemas, and main logic.

<code>services/llm_router/</code>	Intelligent language model router with fallback chain.
<code>services/rag_service/</code>	Retrieval-augmented generation: arXiv for science, Serper+AlphaVantage for finance.
<code>services/image_service/</code>	Image acquisition (Unsplash $\rightarrow$ Hugging Face $\rightarrow$ DALL-E).
<code>services/publisher/</code>	Direct publishing on Twitter/X, Instagram, and LinkedIn.
<code>services/company_profile/</code>	Brand profile management injected into every generation.
<code>services/database/</code>	Connection to PostgreSQL (SQLModel + asynchronous SQLAlchemy).
<code>services/guardrails/</code>	Security and compliance validations for generated content.

<code>data/chroma_db/</code>	Local ChromaDB vector database for scientific RAG.
<code>data/papers/</code>	arXiv PDFs downloaded for indexing in ChromaDB.
<code>data/images/</code>	Local cache of generated images.
<code>infra/</code>	Infrastructure files: Docker, deployment configurations.
<code>main.py</code>	FastAPI application entry point, defines all endpoints.
<code>docker-compose.yml</code>	Container orchestration: API, PostgreSQL, Redis.
<code>pyproject.toml</code>	Project dependencies (managed with Poetry).

3. Entry Point - main.py

The `main.py` file is the visible heart of the system. This is where the FastAPI application is initialized, all endpoints are registered, and all services are connected. It acts as the public "facade" of the multi-agent system.

3.1 Initialization and Lifespan

FastAPI introduces the concept of lifespan to execute code when starting up and shutting down the application. In this project, it is used to connect to the PostgreSQL database and create the necessary tables if they do not exist when the application starts. This ensures that the database is always ready before accepting requests.

Python

```
@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
```

```
    # ON STARTUP: Connect to DB and create tables
```

```
    async with engine.begin() as conn:
```

```
        await conn.run_sync(SQLModel.metadata.create_all)
```

```
    yield
```

```
    # ON SHUTDOWN: Resource cleanup
```

The reason for creating tables during startup instead of in a separate script is that it makes the system more robust for cloud deployments, where the database might not be ready when the developer runs manual scripts.

3.2 CORS (Cross-Origin Resource Sharing)

The CORS middleware is configured to allow requests from development frontends (`localhost:3000` and `localhost:5173`). Without this configuration, browsers would block requests to the API due to security policies. It is allowed locally, but in production, it would be restricted to the project's production domain.

3.3 Main Endpoint: POST /generate

This is the most important endpoint in the system. When a user requests content, this endpoint coordinates the entire pipeline:

Step	Action and Reason
1. LLM Generation	Calls <code>generate_content()</code> which activates the correct agent based on the platform.
2. Image Processing	Extracts the generated image (can be base64 or a URL) and uploads it to Supabase Storage.
3. Conversion to Public URL	Converts the base64 image into a public URL so it can be displayed anywhere.
4. Persistence in PostgreSQL	Saves the generated content, topic, category, and image URL into the database.
5. Response Return	Returns the complete <code>GenerateResponse</code> to the client.

The use of Supabase Storage addresses a practical need: AI-generated images in base64 are too heavy to store in PostgreSQL and are not directly usable by Instagram or LinkedIn, which require public URLs accessible from the internet.

3.4 Endpoint: GET /history

Returns the last 20 generations stored in PostgreSQL, sorted from most recent to oldest. It allows the user to view and reuse previously generated content, avoiding duplicate requests and API costs.

3.5 Publishing Endpoints

The endpoints `/publish/twitter`, `/publish/instagram`, and `/publish/linkedin` act as direct gateways to social platform APIs. The Twitter endpoint includes special logic: it automatically detects if the content is a thread (contains '1/5', '1/', etc.) and uses the `post_thread()` function instead of `post_tweet()`.

4. Content Generator Agent

The `content_generator` module is the intelligent core of the system. It contains three key files: `generator.py` (generation logic), `prompts.py` (instructions for the LLM), and `schemas.py` (表达 contracts).

4.1 generator.py - Generation Logic

The `generate_content()` function implements the Strategy pattern: it dynamically selects the correct generation pipeline based on the requested platform. This decision is critical because different platforms require completely different context data.

Platform	Pipeline Used
science	RAG with ChromaDB + arXiv papers (verified scientific context)
finance	Financial RAG: real-time news (Serper) + market prices (Alpha Vantage)
blog, twitter, instagram, linkedin	Standard prompt with platform-specific instructions

Why differentiate by platform in the generation layer?

Because the context injected into the prompt changes radically depending on the type of content. For science, you need verified academic papers. For finance, you need real-time market data. An LLM without updated context can invent incorrect financial or scientific data; this is called "hallucination" and is especially dangerous in these fields.

Both RAG paths implement a fallback pattern: if retrieving external context fails (network down, API rate limits, etc.), the system degrades gracefully to the standard prompt instead of returning an error to the user.

4.2 schemas.py - Data Contracts

Pydantic schemas define the "contract" between the client and the API: what data can be sent, what values are valid, and what the response will look like. Using enums instead of free strings has two major advantages:

- **Automatic Validation:** FastAPI rejects requests with unsupported platforms or models, showing a clear error message.
- **Automatic Documentation:** Swagger UI shows exactly which values are accepted in each field.

Schema	Purpose
Platform (Enum)	The 6 supported platforms: blog, twitter, instagram, linkedin, science, finance
Language (Enum)	Available languages: Spanish, English, French, Italian
Tone (Enum)	Writing tones: formal, casual, inspirational, educational
ModelChoice (Enum)	Available LLMs: groq, gemini, ollama, haiku (Claude), sonnet (Claude)
GenerateRequest	Body of the POST /generate request with all parameters
GenerateResponse	Response with generated content, metadata, and image URL

4.3 prompts.py - Instructions for the LLM

This file contains detailed instructions that guide the LLM to produce professional-quality content. It is one of the most critical files in the system because the output quality depends directly on the prompt quality. Three configuration dictionaries are defined:

- **PLATFORM_INSTRUCTIONS:** Specific formatting instructions for each platform. For example, for Twitter, it indicates generating 5 numbered tweets with a maximum of 280 characters each, an initial hook, and a final CTA.
- **LANGUAGE_INSTRUCTIONS:** Language instructions directing the LLM to write in a native tone, not as a literal translation.
- **TONE_INSTRUCTIONS:** Instructions that modulate style (formal vs. casual vs. inspirational vs. educational).

The **get_prompt()** function assembles all these instructions into a single structured prompt and optionally injects the **company_context** (brand profile). The **get_science_prompt()** and

`get_finance_prompt()` functions are specialized variants that also include the context retrieved by the RAG system, clearly labeled so the LLM understands it is verified external data and must cite it.

5. LLM Router Agent - `llm_router`

The LLM Router is one of the most strategic components of the system. Its function is to completely abstract the language model layer: the rest of the system never calls OpenAI, Groq, or Gemini directly; it always goes through the router. This provides three key benefits:

- **Provider Independence:** If a provider raises prices or goes down, it can be changed without touching the generator code.
- **Automatic Fallback:** If the preferred model fails, the router automatically tries the next one in the chain.
- **User Selection:** The user can choose the model from the UI, balancing speed, quality, and cost.

5.1 Fallback Chain

The router implements a fallback chain ordered by availability and cost:

```
$$\text{Priority 1: groq/llama-3.3-70b-versatile (Free, very fast)}$$
```

```
$$\rightarrow \text{Priority 2: gemini/gemini-2.0-flash (Free)}$$
```

```
$$\rightarrow \text{Priority 3: cerebras/llama3.1-70b (Free)}$$
```

```
$$\rightarrow \text{Priority 4: ollama/llama3.2 (Local, no API cost)}$$
```

The logic is: attempt the model requested by the user, and if it fails, try the models in the chain until one responds. This ensures that the system will always attempt to generate content even if the primary provider is down.

5.2 LiteLLM - API Abstraction

The router uses the LiteLLM library, which provides a unified interface for more than 100 language models from different providers. Thanks to LiteLLM, the code to call the LLM is identical regardless of whether Groq, Gemini, Anthropic (Claude), or Ollama is used; only the model string changes.

5.3 Traceability with LangSmith

The `get_llm_response()` function is decorated with `@traceable` from LangSmith. This automatically logs every call to the LLM in LangSmith, including the prompt sent, response received, tokens used, and latency. This observability is essential for:

- Detecting prompts that yield low-quality results.
- Monitoring cumulative token costs.
- Comparing the quality of different models in production.

6. RAG System - Retrieval-Augmented Generation

RAG (Retrieval-Augmented Generation) is a technique that improves the quality of content generated by LLMs by injecting external, verified information into the prompt. Instead of the LLM generating content solely from its training (which may be outdated or incorrect), the system first retrieves relevant data and includes it as context.

The project implements two RAG variants: one for scientific content (based on arXiv papers) and another for financial content (based on real-time news and market data).

6.1 Scientific RAG ([retriever.py](#) and [fetcher.py](#))

How does the scientific RAG pipeline work?

Stage	Description
1. Paper Download (fetcher.py)	Searches and downloads PDFs from arXiv using the arxiv library. Saves them in data/papers/ .
2. Indexing in ChromaDB	PDFs are split into text chunks, embeddings are generated with Hugging Face MiniLM, and they are stored in ChromaDB.
3. Retrieval (retriever.py)	When a user requests scientific content, ScienceRetriever converts the topic into an embedding and searches for the most semantically similar chunks in ChromaDB.
4. Prompt Injection	Retrieved chunks are injected into the prompt using get_science_prompt() , providing the LLM with verified academic context.

Why use ChromaDB instead of a simple text search?

Because vector search looks for semantic similarity, not exact word matches. If the topic is "gene therapy in rare diseases", it will find chunks about "CRISPR genome editing applied to inherited disorders" even if they do not share exact words.

ScienceRetriever - Cloud vs. Local Mode

The [ScienceRetriever](#) class supports two modes of operation based on available credentials. If [CHROMA_API_KEY](#), [CHROMA_TENANT](#), and [CHROMA_DATABASE](#) exist, it connects to ChromaDB Cloud. If not, it uses a local [PersistentClient](#) in [data/chroma_db/](#). This design allows for local development without infrastructure costs.

6.2 Financial RAG ([financial_news.py](#))

The financial RAG queries real-time data sources on every request. This is necessary because financial data changes minute by minute.

Source	Data Obtained
Serper API (Google News)	Recent financial news: headline, snippet, date, and source.
Alpha Vantage API	Current stock price, intraday change, volume, and last trading date.

The `get_financial_context()` function combines both sources into a single context block. It also includes a ticker mapper (`TICKER_MAP`) that converts company names into stock symbols (TESLA $\rightarrow$ TSLA, APPLE $\rightarrow$ AAPL).

7. Image Agent - image_service

The image service implements a three-level fallback chain to obtain a relevant image for each generated piece of content. The principle is: try the free, highest-quality option first, and degrade to the next level only if it fails.

Level	Service	Details
Priority 1 (Free)	Unsplash API	Professional photography related to the content.
Level 2 (Free with limits)	HuggingFace SDXL	Stable Diffusion 3 image generation by AI.
Level 3 (Paid final backup)	DALL-E 3 (OpenAI)	High-quality AI image generation.

The `build_image_keywords()` function extracts the first 6 words of the generated content (cleaning markdown) to use them as a search query.

Why three levels instead of just using DALL-E?

Because the cost of DALL-E 3 is approximately \$0.04 per image. If the system generates hundreds of contents, the cost accumulates quickly. Unsplash is completely free (50 req/hour) and provides sufficient professional photographs for most use cases.

8. Publisher Agent - publisher

The `publisher` module contains clients to publish content directly to social platforms through their official APIs. Each platform has its own module because their APIs are entirely different from one another.

8.1 Twitter/X — `twitter.py`

Uses the Twitter API v2 with authentication.

8.2 Instagram — `instagram.py`

Step	Description
Step 1: Create container	POST to <code>/ig_user_id/media</code> with the image URL and caption. Returns a <code>creation_id</code> .
Step 2: Publish container	POST to <code>/ig_user_id/media_publish</code> with the <code>creation_id</code> . This publishes the image.

It also implements `post_instagram_carousel()` to publish up to 10 images in a single post (carousel). It first creates individual containers, then a `CAROUSEL` container that groups them, and finally publishes it.

8.3 LinkedIn — `linkedin.py`

Uses the LinkedIn API v2 with the UGC Posts format. The payload requires a nested JSON structure with specific fields like `lifecycleState`, `shareMediaCategory`, and `visibility`. It also implements `post_linkedin_with_image()` for posts with an image referenced by a public URL.

8.4 Scheduled Publishing — `celery_tasks.py`

The system includes a scheduled publishing system based on Celery and Redis. Celery is an asynchronous task manager that allows executing code in the background and scheduling future executions.

9. Company Profile — `company_profile`

The `company_profile` module allows configuring a brand identity that is automatically injected into every content generation. This is essential for ensuring that all content reflects a consistent voice and style.

9.1 What data does the profile store?

Field	Purpose

name	Company name — always included in the context.
sector	Sector of activity — helps the LLM use appropriate terminology.
tone	Brand communication tone (e.g., "professional and approachable").
audience	Target audience — customizes the level of technicality.
keywords_include	Words that must always appear in the content.
keywords_avoid	Words that must never appear (competitors, negative terms).
is_active	Activates or deactivates the profile without deleting it — useful for A/B testing.

9.2 Injection into the prompt

The `to_context_string()` method converts the profile into a structured text block injected into the generation prompt. Only fields containing values are included.

9.3 Profile API

Endpoint	Function
POST /profile/	Creates or updates the workspace profile.
GET /profile/	Obtains the current profile with its generated <code>context_string</code> .
PATCH /profile/toggle	Activates or deactivates the profile without deleting it.

DELETE /profile/	Deletes the profile from the workspace.
------------------	---

Currently, storage is in-memory (`_profile_store` dict) intended for development. In production, it will connect to the `company_profiles` table in PostgreSQL, whose SQLAlchemy model is already defined.

10. Database Layer

The system uses PostgreSQL as its main relational database. The access layer is implemented with SQLModel (which combines SQLAlchemy and Pydantic) and uses asynchronous operations so as not to block the FastAPI event loop.

10.1 GeneratedContent Model

Field	Type and Purpose
<code>id</code>	Integer, autoincrementing primary key.
<code>topic</code>	String — the topic requested by the user.
<code>category</code>	String — category of the content.
<code>generated_text</code>	Text — the full content generated by the LLM.
<code>image_url</code>	String — URL of the associated image.
<code>created_at</code>	DateTime — creation timestamp to sort the history.

10.2 Asynchronous Connection

The connection uses SQLAlchemy's `AsyncSession`, meaning database operations do not block the server while waiting for a response from PostgreSQL. This is crucial in FastAPI: a block on one request would affect all other concurrent requests.

11. Infrastructure and Deployment

11.1 Docker Compose

The `docker-compose.yml` file orchestrates three containers that must run together:

- **API ([Dockerfile.api](#)):** The FastAPI application with all its services.
- **PostgreSQL:** Relational database for generation history.
- **Redis:** Message broker for Celery (scheduled publishing).

The use of Docker ensures that the development environment is identical to production, eliminating the "it works on my machine" problem.

11.2 Environment Variables (.env)

Variable	Service that uses it
DATABASE_URL	PostgreSQL — connection string.
REDIS_URL	Celery / Redis — connection string.
ANTHROPIC_API_KEY	LLM Router — Claude models (Haiku and Sonnet).
SERPER_API_KEY	Financial RAG — news search.
ALPHA_VANTAGE_API_KEY	Financial RAG — real-time market data.
HF_API_KEY	Image Service — generation with HuggingFace SDXL.
OPENAI_API_KEY	Image Service — generation with DALL-E 3 (fallback).
UNSPLASH_ACCESS_KEY	Image Service — photography search (free).

CHROMA_API_KEY + CHROMA_TENANT + CHROMA_DATABASE	Scientific RAG — ChromaDB Cloud.
LANGSMITH_API_KEY	Traceability — LLM call observability.

12. Full Request Flow

To illustrate how all components work together, here is the complete flow of a typical request: generating a LinkedIn post about artificial intelligence.

Step	Component and Action
1. HTTP Request	Client sends <code>POST /generate</code> with <code>platform='linkedin'</code> , <code>topic='artificial intelligence'</code> , <code>model='groq'</code> .
2. Validation	FastAPI + Pydantic automatically validate the request against the <code>GenerateRequest</code> schema.
3. Agent Routing	<code>generate_content()</code> detects <code>platform='linkedin'</code> $\rightarrow$ uses standard prompt (no RAG).
4. Prompt Construction	<code>get_prompt()</code> assembles: LinkedIn instructions + Spanish language + casual tone + topic.
5. LLM Call	<code>get_llm_response()</code> attempts Groq/Llama 3.3 70B with the complete prompt.
6. Image Generation	<code>generate_image()</code> searches Unsplash using the first 6 words of the content.

7. Upload to Supabase	The image is uploaded to Supabase Storage, and a permanent public URL is obtained.
8. Persistence	<code>GeneratedContent</code> is saved in PostgreSQL with the topic, text, and image URL.
9. Response	Returns <code>GenerateResponse</code> with content, <code>model_used</code> , <code>tokens_used</code> , <code>latency_ms</code> , and <code>image_url</code> .

13. Technology Stack

Technology	Role in the Project
Python 3.12	Main backend language.
FastAPI	Asynchronous web framework for the REST API.
SQLModel + SQLAlchemy	ORM for PostgreSQL interaction (async).
PostgreSQL	Relational database for generation history.
LiteLLM	Unified abstraction for multiple LLM providers.
ChromaDB	Vector database for scientific RAG.
HuggingFace Transformers	Local embeddings (MiniLM-L6-v2) for ChromaDB.
Celery + Redis	Asynchronous task queue for scheduled publishing.

Supabase Storage	Image storage with public URLs.
LangSmith	Observability and traceability of LLM calls.
Docker + Docker Compose	Containers for consistent development and production.
Pydantic v2	Data validation and schema serialization.
httpx	Asynchronous HTTP client for external API calls.

Technical Report: Frontend Integration and Bluesky Publishing Flow

1. Architectural Overview and Configuration

The frontend application is built using React, TypeScript, and Vite. It connects to the backend API using environment variables configured in the `.env` file at the root of the project.

- **API Environment Variable:** The application utilizes `VITE_API_URL=http://127.0.0.1:8000` to establish the base endpoint for all asynchronous HTTP requests sent to the backend services.

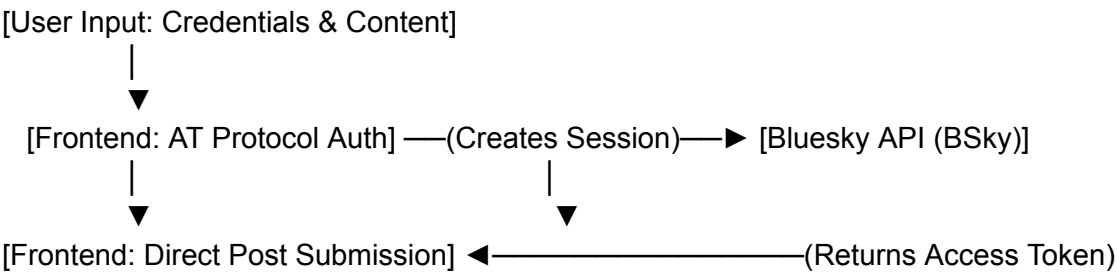
2. Core User Interaction and Chat Workflow

The interface features a chat-based system designed for interacting with AI models, organized within the `src/components/Chat` directory and managed via custom React hooks.

- **Session Management & Input:** The user types prompts into `ChatInput.tsx`. The custom hook `useAgentChat.ts` manages the state of the conversation, handles loading indicators, and dispatches requests to the backend API.
- **Model Responses:** The backend processes the prompt through its multi-agent or LLM routing layers and streams or sends back the response.
- **UI Rendering:** * `ChatWindow.tsx` acts as the primary container for the conversation.
 - `ChatMessage.tsx` renders individual text bubbles for both the user and the AI response.
 - `AgentSteps.tsx` provides visual feedback regarding the execution steps or thought process taken by the backend agents before delivering the final answer.

3. Bluesky Publishing Pipeline

When the AI generates a response suitable for social media, the frontend displays a button allowing the user to publish the content directly to Bluesky. To maintain security and decentralization, the process operates as follows:



Step 1: User Authentication & App Passwords

- **The Interface:** A modal or form layer prompts the user to enter their Bluesky handle (e.g., `user.bsky.social`) and an **App Password** (contraseña de la aplicación).

Security Note: App Passwords are a core feature of the AT Protocol (Bluesky's underlying architecture). They allow third-party applications to post on behalf of a user without exposing their primary master password.

Step 2: Session Creation (Authentication)

- The frontend initiates an asynchronous request (typically via the official `@atproto/api` SDK or a direct HTTP exchange) to the Bluesky repository service endpoint (`com.atproto.server.createSession`).
- **Payload sent:**
- JSON

```
{
  "identifier": "user.bsky.social",
  "password": "xxxx-xxxx-xxxx-xxxx"
}
```

-
-
- **Response received:** Bluesky validates the App Password and returns an access token (`accessJwt`) along with user profile metadata.

Step 3: Resolving and Formatting Content

- The frontend extracts the specific text generated by the AI model from the current `ChatMessage` state.
- If the text contains links or mentions, the frontend uses formatting utilities to calculate the exact character bytes (facets) required by Bluesky to render clickable links properly.

Step 4: Creating the Record (Publishing)

- Using the active session token, the frontend sends a authorized `POST` request to the `com.atproto.repo.createRecord` endpoint.
- **Payload structure:**
- JSON

```
{
  "repo": "user.bsky.social",
  "collection": "app.bsky.feed.post",
  "record": {
    "text": "Content generated by the AI agent...",
    "createdAt": "2026-06-15T10:19:20Z"
  }
}
```

}

-
-
- Once Bluesky returns a status code **200 OK** along with the post URI, the UI updates to show a success message to the user, confirming the post is live.